

HOW TO: CRISPR-NGS ANALYZER

By Lucía de Lamadrid

1) Upload FASTQ (zip) & set run parameters

FASTQs Files:

Browse... Use the browse button or drag and drop a file here

Naming convention for the folders: Day<day>_<gen>_<variant>_<treatment>_<replicate>_[rerun]...

Amplicon Sequence: Paste the amplicon sequence here...

Coding Sequence: Paste the coding sequence here...

Guide Sequence: Paste the guide sequence here...

Minimum alignment score: 50

Plot window size: 20

Run CRISPResso2 on uploaded FASTQs

2) Analyze CRISPResso Results

Enter the MUT sequence: Paste the MUT sequence here...

Enter the WTM sequence: Paste the WTM sequence here...

Results table

FrameShift% Plot

Sensitivity Plot

1 General Considerations

The [CRISPR-NGS Analyzer](#) application is developed to standardize and automate the processing of sequencing data generated from CRISPR-Select experiments. The main goal is to provide a reproducible workflow for analyzing CRISPR-Select datasets without manually running CRISPResso2 for each sample or processing the resulting output files separately.

1.1 Input

The application accepts a **single compressed ZIP archive** containing sequencing files, typically in paired-end FASTQ.GZ format (2 files per sample: R1 & R2).

Each FASTQ filename must follow the naming convention expected by the application

Day<day>_[gen]_<variant>_<treatment>_<replicate>_[rerun]...

so that the following metadata can be extracted automatically:

- Experimental day
- Optional target gene
- Variant identifier
- Treatment or experimental condition
- Biological or technical replicate
- Optional rerun status
- Read direction, such as R1 or R2

The user must also provide the sequence information required by CRISPResso2, including:

- Amplicon sequence
- Guide RNA sequence

- Coding sequence
- Mutant reference sequence
- Wild-type reference sequence

Additional CRISPResso2 parameters, such as the minimum alignment score and plotting window size, can be configured through the application interface.

1.2 Processing Workflow

The application workflow is conceptually divided into two main stages:

Stage 1: CRISPResso2 execution and input processing

1. Extracts the uploaded ZIP archive into a temporary working directory.
2. Identifies and validates FASTQ files.
3. Extracts sample metadata from the filenames.
4. Organizes files by replicate, experimental day, and treatment.
5. Pairs forward and reverse reads.
6. Executes CRISPResso2 independently for each sample.
7. Stores the complete CRISPResso2 output under the output directory.

Stage 2: Post-processing and data analysis

8. Parses the relevant CRISPResso2 result files.
9. Uses the provided mutant and wild-type reference sequences to extract and structure the relevant information from the CRISPResso2 output tables.
10. Calculates metrics, including frequencies and sensitivity.
11. Combines results across selected replicates and experimental conditions.
12. Displays the processed results as tables and plots.
13. Provides the consolidated dataset as a downloadable CSV file.

1.3 Outputs

The application produces the following outputs:

- A summary table containing the calculated metrics for each sample.
- Comparative plots showing:
 - Indel frequencies
 - Frameshift and in-frame mutation frequencies
 - Relative treatment sensitivity
- A downloadable CSV file containing the processed results.
- The complete CRISPResso2 output directory for each analyzed sample.
- Runtime messages indicating sample detection, execution status, warnings, and failures.

2 Repository Structure

The application is developed using Shiny for Python and containerized using Docker.

2.1 app.py

The `app.py` file is the main entry point of the application. It defines both the Shiny user interface and the server-side reactive logic.

The **user** interface is implemented through the `app_ui` object. It contains the layout and controls required to upload the input file, provide sequence information and parameters, select replicates, define the reference treatment, and reorder treatments for visualization.

The **server** function coordinates the complete application workflow. It is divided into three main steps: input preparation, CRISPResso2 execution, and result post-processing. Here, all functions from the `modules/` folder are called.

2.2 modules

- **runner.py:** contains the functionality responsible for discovering sequencing files, pairing forward and reverse reads, and executing CRISPResso2 in parallel.
- **parser.py:** contains the low-level parsing functions used to extract numerical values from individual CRISPResso2 output files. The module currently provides functions to retrieve:
 - The total number of aligned reads.
 - Modified and unmodified read counts.
 - Frameshift and in-frame mutation counts.
 - Mutant and WT* read counts from the CRISPResso2 allele-frequency table.
It compares each aligned sequence with the user-provided mutant and WT* reference sequences. The allele with the longest matching substring is selected independently for each reference sequence. The sequences must be valid, containing only [ATCGN-] characters.

Note: This module assumes specific CRISPResso2 column names and text structures. Changes to CRISPResso2 output formats, metric definitions, allele-classification logic, or sequence-matching methods should therefore be implemented here.

- **utils.py:** provides the high-level utility functions that connect input handling, directory organization, CRISPResso2 output discovery, parsing, and DataFrame construction.
 - **extract_zip_to_tmp():** extracts the uploaded ZIP archive into a Python TemporaryDirectory.
 - **parse_fastq_metadata():** applies a regular expression to each FASTQ filename and extracts the experimental day, optional gene identifier, variant, treatment, replicate number, and optional rerun status.
 - **reorganize_extracted_fastqs():** moves the FASTQ files into a standardized internal directory structure.
 - **list_replicates_from_outputs()** and **list_treatments_from_outputs():** inspect the output directory after CRISPResso2 execution and return the replicate and treatment folders available for downstream analysis.

- **`process_treatment_folder()`**: locates the required CRISPResso2 result files for an individual sample. It calls the functions defined in *parser.py* and combines the extracted values into a single dictionary.
- **`calculate_sensitivity()`**: calculates relative sensitivity by normalizing the MUT/WT*% of each treatment against a user-selected reference treatment.
- **`build_dfs_by_replicate()`**: processes all detected treatment directories, creates one pandas DataFrame per replicate, adds the sensitivity values, and returns the results as a dictionary indexed by replicate name.
- **`plots.py`**: contains the Matplotlib-based visualization functions used by the Shiny application. The **frameshift and indel plots** are grouped by treatment, with separate bars for each replicate, to visualize how editing outcomes vary between experimental conditions and assess their reproducibility across replicates. The **sensitivity plot** is grouped by replicate, with separate bars for each treatment, to visualize and compare the relative treatment response within each replicate.

2.3 Docker containerization

The CRISPR-NGS Analyzer application runs in a Docker container, so you do not need to install any dependencies manually. The **Dockerfile** defines the complete runtime environment required to execute the application:

- The build process installs mamba and then uses the Conda Forge and Bioconda channels to install CRISPResso2 and its external command-line dependencies.
- Python dependencies are installed separately from requirements.txt using *pip*.
- The container exposes port 8000 and starts the application.

compose.yaml defines the local Docker Compose service used to build and run the application. The configuration creates a service named *web*, builds the image using the Dockerfile in the current repository, and maps *port 8000* of the host machine to *port 8000* inside the container.

3 Local Development Setup

The application is currently deployed on the Rigshospitalet server under the supervision of Mauricio Moldes. Any code changes should first be developed and tested locally. To do so, clone or copy the repository to your local computer, and use Docker to build and run the application in an environment equivalent to the deployed version.

1. Make sure you have Docker installed and running on your computer.

```
docker --version
```

```
docker compose version
```

2. Clone the repository

```
git clone https://github.com/luciadelama/CRISPR-NGS_analyzer.git
```

```
cd CRISPR-NGS_analyzer
```

3. Start the app

```
docker compose up --build
```

The first build may take some time because the Docker image installs the required bioinformatics tools and Python dependencies.

4. Open the app

Once the container is running, open your browser and go to: <http://localhost:8000>

Note: after each code change that needs to be tested, the running container must be stopped, and the image rebuilt before the changes become visible.

Stop and remove the current container with: *docker compose down*

Then rebuild and restart the application: *docker compose up --build*